



Tema 04. Programación orientada a objetos

Parte 2. Excepciones





Índice

1. Introducción
2. Excepciones en Java
3. Propagación de excepciones
4. Manejo de excepciones
5. La clase Exception
6. Tipos de excepciones

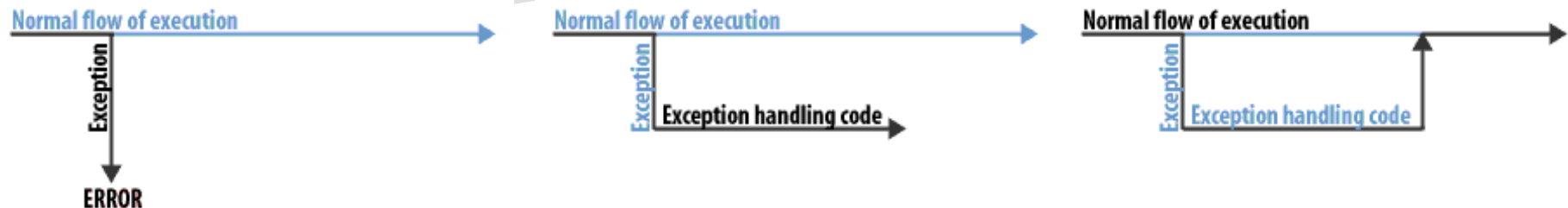


1.- Introducción.

Una **excepción** es un evento que ocurre durante la ejecución de un programa, interrumpiendo el flujo normal de las instrucciones.

Las excepciones suelen estar asociadas a errores en **tiempo de ejecución**.

Pueden tener distintas causas / orígenes, es nuestra labor detectarlas y tratarlas para que no alteren el correcto funcionamiento del programa.

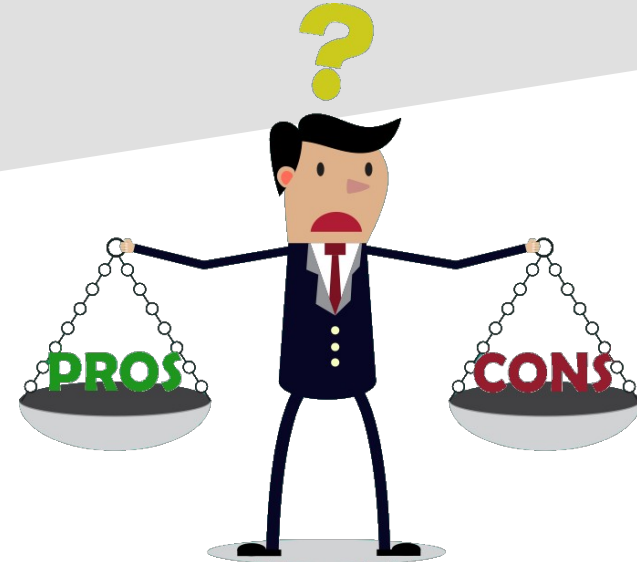




1.- Introducción.

Ventajas de usar excepciones:

- Separar el código general del proceso de errores.
- Propagar errores hasta encontrar el manejador adecuado.
- Agrupar y diferenciar tipos de error.
- Mejor legibilidad del código.
- ...





2.- Excepciones en Java.

Java trata las excepciones como **objetos** (clase `Exception`).

Cuando se produce un error en tiempo de ejecución, se crea un objeto (de tipo `Exception`) con información relativa al error:

- Tipo de excepción.
- Estado del programa.

Una vez creada la excepción pueden ocurrir dos cosas:

- El programa se detiene y muestra el error (comportamiento **no** deseado).
- Existe código preparado para gestionar el error (comportamiento deseado).

*El código destinado a detectar y tratar excepciones se denomina **manejador (handler)**.*



2.- Excepciones en Java.

EJEMPLO: división de un número entre 0, excepción no controlada.

```
public static void main(String[] args) {  
    int x = 2, y = 0, res;  
    res = x / y;  
    System.out.println(res);  
}
```



```
<terminated> Ejemplos [Java Application] C:\Users\y0rg\eclipse\plugins\org.eclipse.justj.openjdk.hotspot.jre.full.win32.x86_64_16.0.1.v20210528-1205\jre\bin  
Exception in thread "main" java.lang.ArithmeticException: / by zero  
    at Ejemplos.main(Ejemplos.java:6)
```



3.- Propagación de excepciones

Las excepciones se generan en el ámbito de ejecución en el que se encuentra el programa. Puede ser el método principal (`main`) o en un método de una clase.

Cuando se produce una excepción, puede ser tratada en el mismo método que la genera o **propagarse** hacía otros métodos “superiores” hasta llegar al `main`.

Ejemplo: *excepción generada en el método `floorDiv` que llega al método `main`*

```
public static void main(String[] args) {  
    int x = 2, y = 0;  
    Math.floorDiv(x, y);  
}
```

```
<terminated> Ejemplos [Java Application] C:\Users\y0rg\eclipse\plugins\org.eclipse.justj.openjdk.hotspot.jre.full.win32.x86_64_16.0.1.v20210528-1205\jre\bin  
Exception in thread "main" java.lang.ArithmeticException: / by zero  
    at java.base/java.lang.Math.floorDiv(Math.java:1169)  
    at Ejemplos.main(Ejemplos.java:9)
```



4.- Manejo de excepciones

Cuando se produce una excepción se puede manejar de dos formas:

- Tratarla directamente.
- Lanzarla al método inmediatamente “superior” (**solo si no es el main**).

Si un método lanza una excepción, hay que indicarlo en su definición mediante la palabra reservada **throws** seguida del tipo de excepción.

```
public double division (int x, int y) throws ArithmeticException{
```

Java tiene definidas una serie de excepciones “estandar”, pero también podemos definir las nuestras propias.



4.- Manejo de excepciones

Las excepciones se capturan en bloques `{ } try – catch – finally`.

- **try** `{ ... }` incluye las instrucciones que pueden generar la excepción.
- **catch** `( TipoExcepcion nomVarExcp ) { ... }` sigue al bloque `try`, contiene las instrucciones que se ejecutarán en caso de generarse la excepción. Se debe crear un bloque `catch` por cada tipo de excepción que puede ser generada en el bloque `try`.
- **finally** `{ ... }` instrucciones que se ejecutarán independientemente de que se genere o no la excepción.

Gotta catch 'em all!





4.- Manejo de excepciones

EJEMPLO: bloques try – catch - finally

```
try {  
    //Instrucciones que pueden fallar  
}  
catch (TipoExcepcion1 nom_variable) {  
    //Instrucciones a ejecutar si se produce la excepción1  
}  
catch (TipoExcepcion2 nom_variable) {  
    //Instrucciones a ejecutar si se produce la excepción2  
}  
//...  
finally {  
    //Instrucciones a ejecutar siempre  
}
```



4.- Manejo de excepciones

EJEMPLO: bloques try – catch de la división por 0.

```
public static void main(String[] args) {  
    int x = 2, y = 0, res = 0;  
    try {  
        res = x / y;  
    }  
    catch (ArithmeticException excepcion) {  
        res = Integer.MAX_VALUE; //Por ser infinito  
    }  
    finally {  
        System.out.println(res);  
    }  
}
```

NOTA: a pesar de la excepción, el programa finaliza correctamente.



5.- La clase Exception

Todas las excepciones en Java heredan (comparten) características de una clase genérica de excepciones, la clase `Exception`.

Todas las excepciones se pueden propagar, lanzar y capturar. Comparten el método `printStackTrace` que muestra información de la pila de llamadas que provocó la excepción (lista de métodos de clases y `main`).

Las excepciones del tipo `RuntimeException` no estamos obligados a capturarlas y tratarlas, el resto de excepciones **SI**, y si no lo hacemos el entorno nos indicará un error de compilación.

Podemos crear clases excepciones que hereden de `Exception` y lanzarlas manualmente cuando creamos conveniente con la palabra reservada `throw`.

```
throw new TipoExcepcion();
```



5.- La clase Exception

EJEMPLO: Creación y lanzamiento de excepciones propias

```
public static void main(String[] args) {  
    int x = 2, y = 0, res = 0;  
    try {  
        res = division(x, y);  
    } catch (DivisionEntreCeroExcepcion excepcion) {  
        System.out.println(excepcion);  
    } finally {  
        System.out.println(res);  
    }  
}
```

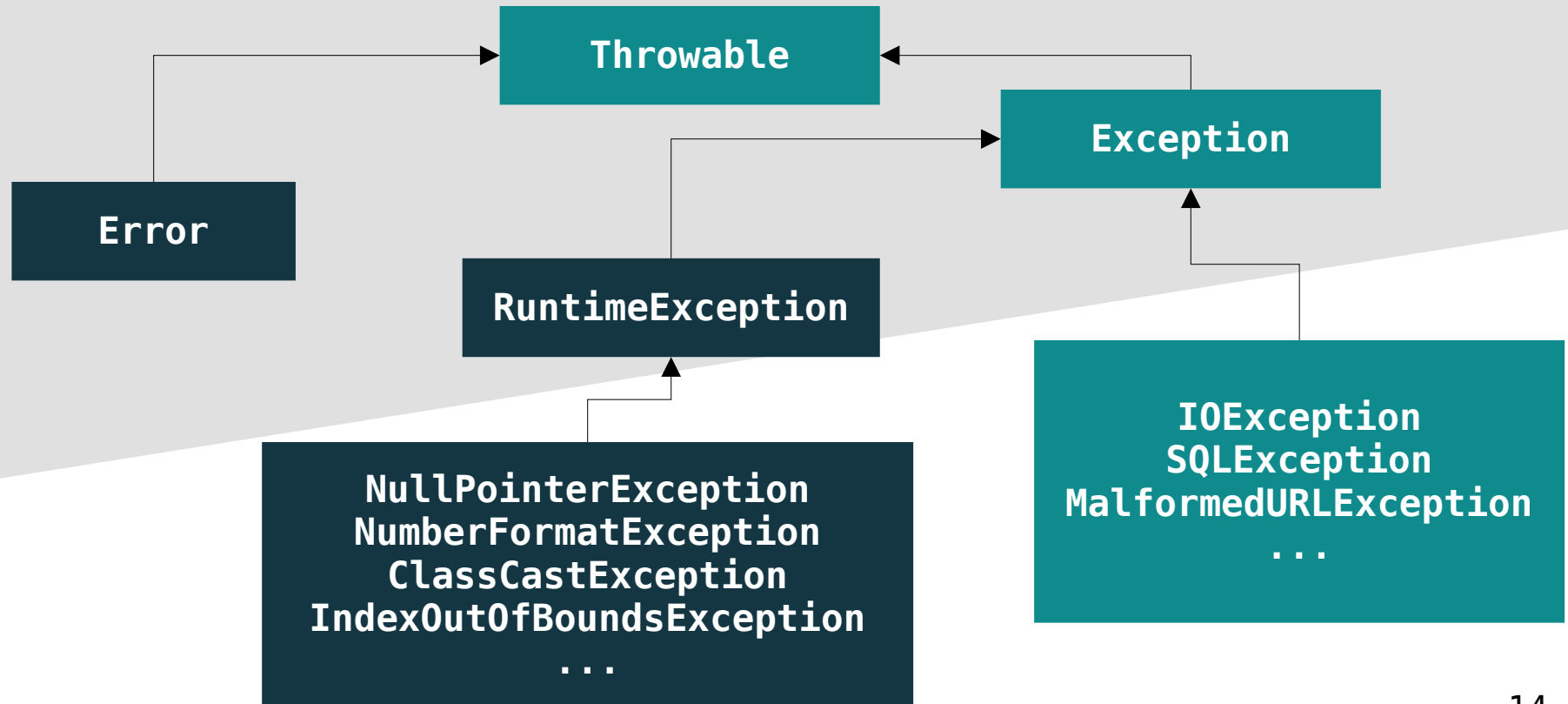
```
public class DivisionEntreCeroExcepcion extends Exception {  
    public DivisionEntreCeroExcepcion () {  
        super("Infinity");  
    }  
}
```

```
public static int division(int x, int y) throws DivisionEntreCeroExcepcion {  
    int res = 0;  
    if (y == 0) {  
        throw new DivisionEntreCeroExcepcion();  
    } else {  
        res = x / y;  
    }  
    return res;  
}
```



6.- Tipos de excepciones

Algunas de las excepciones más comunes son:





Tema 04. Programación orientada a objetos. Excepciones

